

5. Remote Procedure Call

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

5.1 Vorschau

- Grundlagen
- Fehler
- REST
- gRPC
- Zusammenfassung

5.2 Grundlagen

Motivation

- Remote Procedure Call (RPC) als Oberbegriff = Fernaufruf
 - Ursprünglich nur Funktionen ohne Objekte
 - Heute objekt-orientiert und somit entfernte Methodenaufrufe
- Ziel: Transparenz
 - Ein Fernaufruf soll so aussehen wie ein lokaler Funktionsaufruf
 - Parameter und Ergebnis soll automatisch übertragen werden
 - Möglichst sprach- und plattformunabhängig
 - Somit einfacher, als Socket-Programmierung

Motivation

- Fernaufrufe werden häufig verwendet
 - Google gRPC
 - Facebook / Apache Thrift
 - Twitter Finagle
 - ...
- Ursprüngliche Idee von 1984
 - Andrew D. Birrell and Bruce Jay Nelson, „Implementing remote procedure calls“, ACM Transactions on Computer Systems, 2(1):39–59, 1985.

Herausforderungen

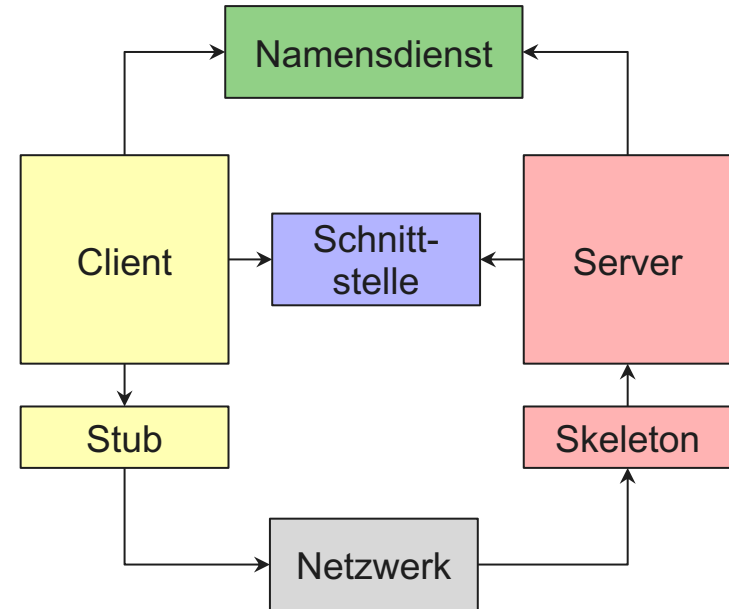
- Zusätzliche Fehlerquellen (gegenüber lokalem Aufruf):
 - Was passiert wenn eine Netzwerknachricht verloren geht?
 - Was ist, wenn der Client, Server oder das Netzwerk ausfallen?
- Geschwindigkeit:
 - Ein Fernaufruf ist langsamer wie ein lokaler Aufruf
 - Lokal: $<10\text{ns}$
 - Entfernt:
 - $<10\text{-}1000\mu\text{s}$ im LAN (in einem Data-Center)
 - $<10\text{-}100\text{ms}$ im WAN (im Internet)

Herausforderungen (2)

- Heterogenität
 - Client und Server verwenden u.U. unterschiedliche Programmiersprachen
 - Datentypen haben daher unterschiedliche Byte-Größen
 - Unterschiedliches Alignment von Daten (4 vs 8 Byte)
 - Eventuell andere Byte-Order (Big vs Little Endian)

Übersicht der Fernaufruf-Komponenten

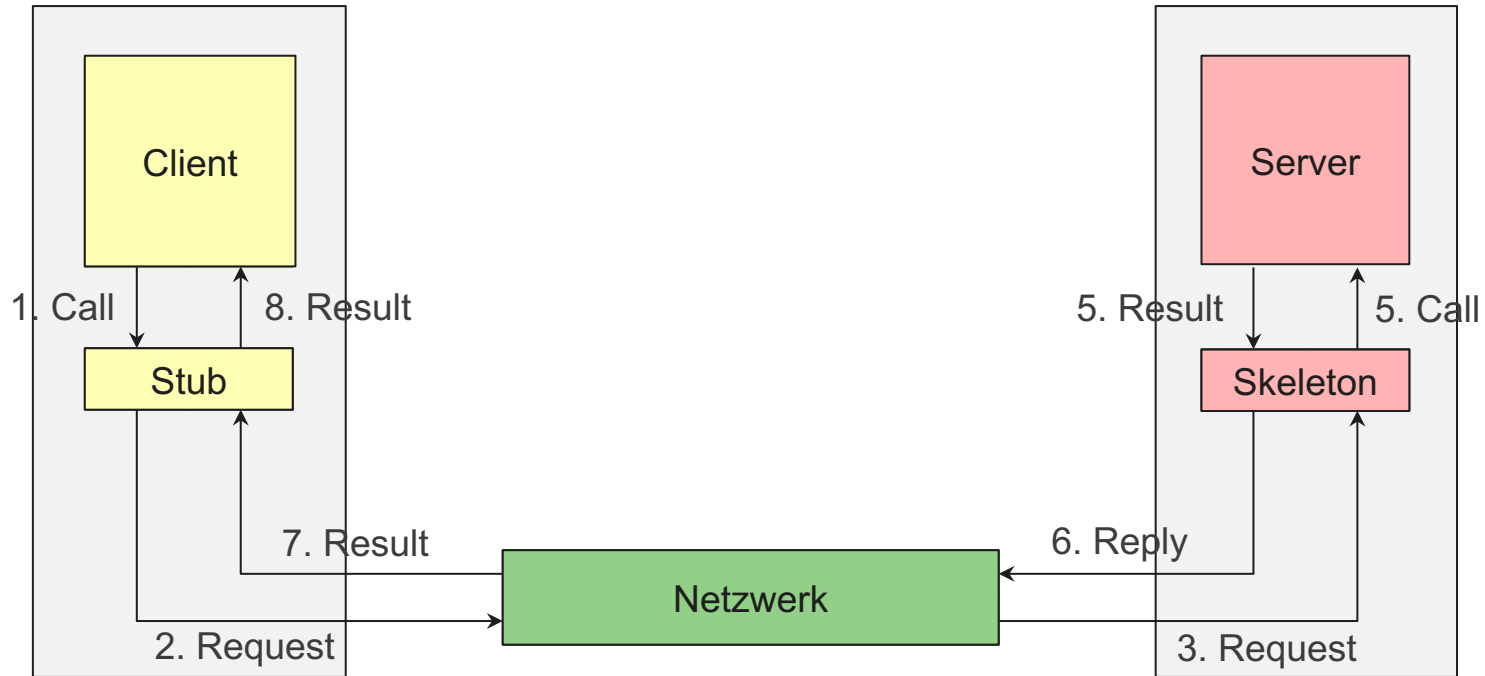
- Schnittstelle: sprach- und plattformunabhängige Beschreibung
 - Bietet fernaufrufbare Funktionen an
- Server
 - Ruft Funktionen auf dem Server auf
- Client
 - Ruft Funktionen auf dem Server auf
- Stub/Skeleton
 - Automatisch generierter Code, welcher die Kommunikation für die Fernaufrufe realisiert
- Namensdienst (optional)
 - Server macht Dienst bekannt
 - Client kann Dienste dort finden



Stub und Skeleton

- Stub (Client)
 - Stellvertreter des Servers auf Client-Seite
 - Umsetzung des Aufrufs in einen Nachrichtenaustausch
 - Verpacken der Aufrufparameter und Entpacken des Rückgabewerts
- Skeleton (Server)
 - Stellvertreter des Clients auf Server-Seite
 - Umsetzung der Aufrufrückkehr in einen Nachrichtenaustausch
 - Entpacken der Aufrufparameter und Verpacken des Rückgabewerts
- Oft gibt es auch andere Namen, beispielsweise Proxy, Stellvertreter, ...

Stub und Skeleton



(Un)-Marshalling der Daten

- = Basis für eine Interaktion per Nachrichtenaustausch
 - **Marshalling**: Umwandlung der zu sendenden Daten in ein Nachrichtenformat
 - **Unmarshalling**: Rekonstruktion der Daten beim Empfänger
- Nachrichteninhalte bei Fernaufrufen
 - Anfrage
 - Bei Methodenfernaufruf: Bezeichner des entfernten Objekts
 - Bezeichner der aufzurufenden Prozedur / Methode
 - Parameter
 - Antwort
 - Normalfall: Rückgabewert(e)
 - Fehlerfall: Informationen zur Fehlersituation (z. B. Exception)

Parameter: Arten

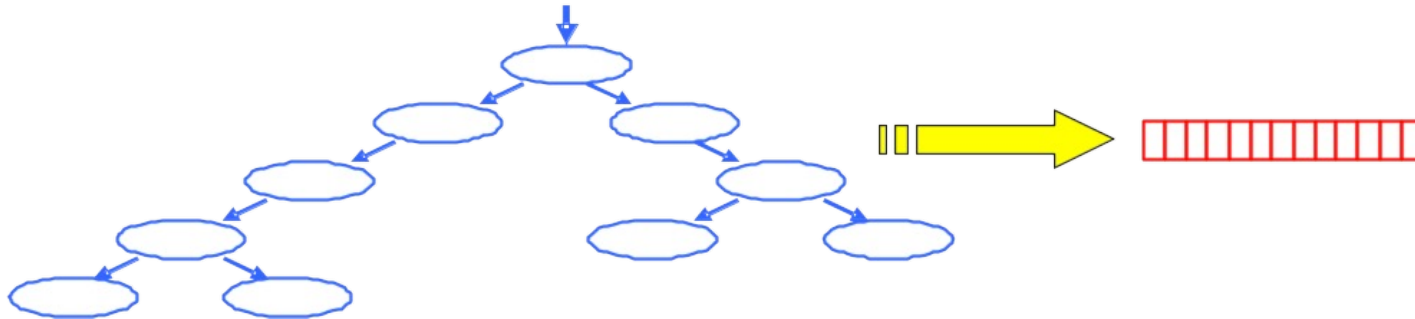
- Eingabeparameter
 - Informationsfluss: Client $\rightarrow$ Server
 - Bestandteil der Anfrage: Call-by-Value
- Ausgabeparameter
 - Informationsfluss: Client $\leftarrow$ Server
 - Bestandteil der Antwort: Call-by-Result
- Ein-/Ausgabeparameter
 - Informationsfluss: Client $\longleftrightarrow$ Server
 - Bestandteil beider Nachrichten: Call-by-Value-Result
 - Ersetzen des Werts auf Client-Seite durch den Wert in der Antwort
- Abhängig von der Programmiersprache (teilweise keine Unterscheidung)

Parameter: Referenzen

- Problem: Umsetzung von **Call-by-Reference**
 - Client und Server haben keinen gemeinsamen Speicher
 - Versand von Zeigern in Anfragen / Antworten geht nicht!
- Lösung 1: Übermittlung einer Kopie des referenzierten Objekts/Struct
 - Problem ein Objekt/Struct kann wiederum eine Referenz beinhalten usw.
 - Daher müssen alle erreichbaren Objekte/Structs übertragen werden
- Lösung 2: Realisierung mittels Remote-Referenz
 - Übermittlung eines systemweit eindeutigen Zeigers anstatt der Speicheradresse
 - Dann erfolgt der Zugriff auf den Parameter wiederum per Fernaufruf

Parameter: Referenzen (2)

- Werden Kopien für eine Referenz übertragen, so müssen auch Referenzen in den Objekten beachtet werden
- Verzeigte Strukturen werden serialisiert ("flachgeklopft") und komplett übertragen → transitive Hüllenbildung
- Wird beispielsweise die Referenz auf die Wurzel eines Binärbaumes übergeben, so wird der gesamte Baum übertragen (u.U. sehr aufwändig)
→



Automatische Generierung von Stubs und Skeletons

- Durch den Compiler oder zusätzliche Werkzeuge
- Variante 1: Annotation von fernaufrufbaren Methoden
 - Beispiel: WebServices
- Variante 2: explizite Schnittstellenbeschreibung in einer separaten Datei
 - Beispiel: Interface Definition Language (IDL)
 - RPC: C-Syntax
 - Corba: C++-Syntax
 - Mit zusätzlichen Attributen in, out (um Parameterübertragung zu optimieren)

5.3 Fehler

Gutartige Fehler

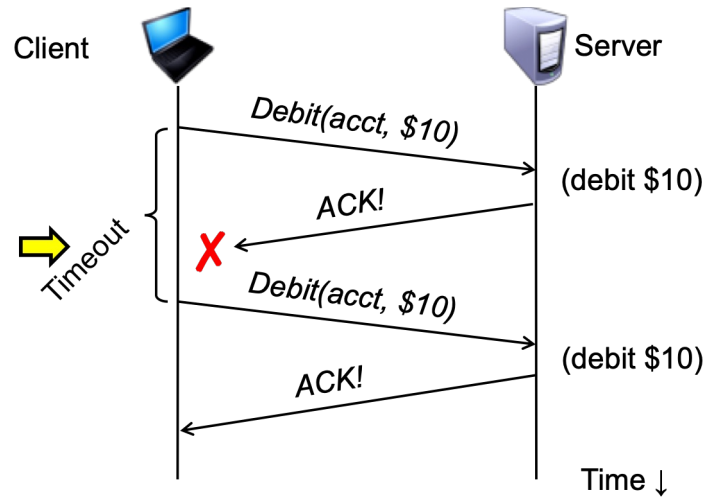
- Was kann alles schief gehen?
 - Client-Crash
 - Server-Crash
 - Paketverlust
 - Verzögerungen (Netzwerk und/oder Server können zeitweise sehr langsam sein)
- Problem für den Client
 - Er kann die Ursache eines fehlenden Ergebnisses nicht erkennen
- Lösung: falls keine Antwort (nach einer bestimmten Zeit) ankommt, Fernaufruf wiederholen
 - Neues Problem: vielleicht wurde der Aufruf schon erfolgreich durchgeführt und nur die Antwort ging verloren

At-Least Once

- Semantik: Fernaufruf wird auf dem Server mindestens ein Mal durchgeführt, u.U. also mehrfach
- Client-Stub wartet bestimmte Zeit auf eine Antwort vom Server
- Falls in dieser Zeit keine Antwort vom Server eingeht, wird der Aufruf wiederholt
- Dies wird einige Male gemacht und falls es dann immer noch nicht klappt, wird an die Anwendung ein Fehler gemeldet

At-Least Once - Problem

- Gute geeignet, falls auf dem Server nur Daten gelesen werden
- Aber problematisch falls auf dem Server geschrieben wird und es Seiteneffekte gibt
 - Beispiel: hier würden 2x \$10 abgebucht



At-Most Once

- Semantik: Fernaufruf wird auf dem Server höchstens ein Mal durchgeführt (u. U. kein einziges Mal)
- Server-Skeleton erkennt wiederholte Fernaufrufe und schickt direkt die alte Antwort, ohne den Aufruf erneut durchzuführen
- Damit wiederholte Fernaufrufe erkannt werden können muss der Client diese um eine global eindeutige ID erweitern und der Server speichert diese ID mit seiner Antwort

At-Most Once - Probleme

- Wie lange muss der Server alte Antworten und IDs speichern?
- Client kann dem Server mitteilen, dass er die Antwort bekommen hat
 - Optimierung: „piggyback“ mit nächstem Fernaufruf
 - Aufwändig, falls viele parallele Fernaufrufe stattfinden
- Was ist wenn der Server abstürzt?
- Die alten Antworten und IDs müssen vorsorglich auf Disk gespeichert werden, damit diese beim Neustart geladen werden können
 - Ansonsten würde der Server Duplikate nicht erkennen und Aufrufe doch mehrfach durchführen

Exactly Once

- Fernaufruf wird auf dem Server genau ein Mal ausgeführt
- Benötigt:
 - Wiederholung von Fernaufrufen wie bei At-Least Once
 - Sowie die Filterung von Duplikaten wie bei At-Most Once
 - Zusätzlich muss der Client offene Fernaufrufe verwalten und auf Disk speichern, damit er bei einem Absturz diese erneut (mit der gleichen ID) wiederholen kann
 - Server muss: bearbeitete Anfragen und Ergebnisse persistent speichern und nach Abstürzen korrekt wiederherstellen
- Sehr aufwändig und kaum verwendet in der Praxis

5.4 REST

Representational State Transfer (REST)

- Intention: Entwicklung eines Regelwerks für verteilte Systeme zur Bereitstellung von Anwendungen und Diensten über das Internet

„The name “**Representational State Transfer**” is intended to evoke an image of **how a well-designed Web application behaves**: a network of webpages (a virtual state-machine), where the user progresses through the application by selecting links (state transitions), resulting in the next page (representing the next state of the application) being transferred to the user [...]”

R. Fielding, „Architectural Styles and the Design of Network-based Software Architectures“, Dissertation, 2000.

- Architekturkonzept
 - Sammlung von Grundprinzipien, keine konkrete Implementierung
 - Anwendungsbeispiel: World Wide Web (WWW), siehe später

Zentrale Konzepte

1. Aufteilung in Client und Server
2. Zustandslose Interaktion
3. Ressourcen & Repräsentationen

1. Aufteilung in Client und Server

- **Server:** Bereitstellung eines Dienstes und Verwaltung von Daten
- **Client:** Nutzung von Diensten durch Senden von Anfragen an den Server
- Vorteile
 - Beide Seiten können unabhängig voneinander weiterentwickelt werden, solange die Schnittstelle zwischen ihnen gleich bleibt.
 - Erhöhte Portabilität der Client-Seite, auch in heterogenen Umgebungen
- Nachteile
 - Server-Seite kann bei schlechter Skalierbarkeit zum Flaschenhals werden

2. Zustandslose Interaktion

- Zustandslose Kommunikation über HTTP → Anfragen müssen den notwendigen Zustand jeweils mitsenden
 - Server merkt sich nichts über einen Client
- Vorteil
 - Einfachere parallele Bearbeitung von Anfragen → bessere Server-Skalierbarkeit
 - Vereinfachte Lastverteilung durch Load-Balancer
 - Einfache und schnelle Fehlererholung, falls der Server abstürzt
- Nachteil
 - Mehraufwand durch Mitsenden von Kontextinformationen

3. Ressource & Repräsentation

- **Identifikation von Ressourcen** mithilfe Universal Resource Identifiers (URIs)
 - Beispiel `http://www.boeing.com/aircraft/747`
 - Ressource als abstraktes Konzept, unabhängig von der konkreten Darstellung
 - Beispiel: Flugzeug identifiziert über URI
- Ressourcen können unterschiedliche **Repräsentationen** haben
 - Beispiel: Repräsentation eines Buchs: PDF-Datei oder Taschenbuch

Beispiel für RESTful System: World Wide Web

- Das WWW ist ein klassisches Beispiel eines REST-basierten Systems.
 - REST wurde aus der Analyse der Architektur des WWW abgeleitet,
 - Zugriff auf Ressourcen (Webseiten, Bilder, Videos usw.)
 - **Identifikation** von Ressourcen über **URIs**
 - **Repräsentationen** sind konkrete Darstellungen einer Ressource (z. B. HTML, GIF oder MP4)
 - **Navigation** innerhalb der Anwendung erfolgt über **Hyperlinks**
 - **HTTP als Anwendungsprotokoll**
 - Einhaltung der spezifizierten Semantiken der HTTP-Requests
 - GET und HEAD sind rein lesende Methoden
 - POST, PUT und DELETE sind modifizierende Methoden
 - Antworten im Status-Code der HTTP-Antworten
 - 200: OK
 - 404: Not Found, ...

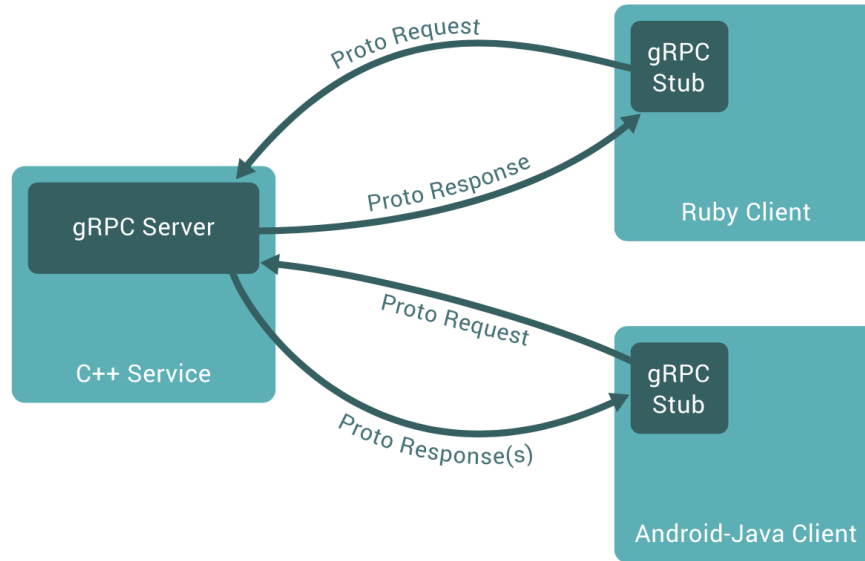
5.5 gRPC



- Entwickelt von Google
- Plattform- und sprach-unabhängiger RPC
- Basiert auf [protocol buffers \(protobuf\)](#) für die sprachunabhängige Definition von Datentypen und Schnittstellen (neueste Version proto3)
- Verwendet als Transport HTTP/2
 - Erlaubt Multiplexing mehrerer Streams über eine TCP-Verbindung.
 - HTTP1 öffnet ca. 4 – 8 Verbindungen für einen Client
- Sicherheit über SSL/TLS
- Open-Source seit 2015: <https://grpc.io>
- Verwendet in vielen Unternehmen, u.a. auch für Microservices

Übersicht

- Protocol Buffers bietet eine Interface Definition Language (IDL) und definiert die Schnittstellen und Nachrichten
- Unterstützt viele gängigen Sprachen : Java, Python, C#, C++, ...



Protocol Buffer Messages

- Messages dienen der Serialisierung strukturierter Daten
- Der Inhalt wird in einer Text-Datei mit Endung **.proto** definiert:
 - Strukturierte Nachrichtentypen: Typ, Name, Feldnummer
 - Beispiel:

```
message Person {  
    string name = 1;  
    int32 id = 2;  
    bool has_ponycopter = 3;  
}
```
 - Mithilfe des protocol buffer compilers **protoc** können für die gewünschte Programmiersprache Zugriffsklassen generiert werden
 - Beinhalten getter- und setter-Methoden
 - Sowie Methoden zum De-/Serialisieren der Nachrichten

Protocol Buffer Service

- Als nächstes muss ein Service in einer **.proto**-Datei definiert werden
 - Dieser besteht aus Methoden, welche für die Typen der Parameter und dem optionalen Rückgabewert die Protocol Buffer Messages verwenden
- **protoc** generiert client/server code (stub und skeleton)

```
service Greeter {  
    rpc SayHello (HelloRequest) returns (HelloReply) {}  
}  
  
message HelloRequest {  
    string name = 1;  
}  
  
message HelloReply {  
    string message = 1;  
}
```

Protocol Buffer Service (2)

- Neben den „normalen“ Methoden-Aufrufen (ein Request und eine Response), gibt es auch die Möglichkeit Messages als Datenstrom zu übertragen
 - Der Datenstrom besteht aus einer Sequenz von Messages
 - Dies geht uni- oder bi-direktional
- Beispiel:

```
service Greeter {  
  rpc SayHello (HelloRequest) returns (HelloReply) {}  
  rpc LotsOfReplies (HelloRequest) returns (stream HelloReply) {}  
  rpc LotsOfGreetings (stream HelloRequest) returns (stream HelloReply) {}  
  rpc BidiHello (stream HelloRequest) returns (stream HelloReply) {}  
}
```

Weitere Eigenschaften

- Es gibt sowohl **synchrone** als auch **asynchrone** RPC-Aufrufe
- **Deadlines**: Client kann eine Frist angeben, bis wann ein RPC abgeschlossen sein muss → bei Ablauf terminiert der Client den Aufruf lokal
 - Falls die Antwort des Servers verspätet ankommt wird diese verworfen
- **Abbruch**: Server und Client können einen RPC-Aufruf jederzeit abbrechen
- **Metadata**: Sicherheits-Informationen (Authentisierung) für einen RPC-Aufruf

- Authentisierung pro Channel (Verbindung) oder für einen RPC-Aufruf, beispielweise mit SSL/TLS
- TLS: Transport Layer Security
 - Nachfolger von SSL
 - Standardprotokoll zur Absicherung von Netzwerkkommunikation
 - Basiert auf Zertifikaten, siehe letztes Kapitel
 - Neueste Version TLS 1.3, 2018
- SSL: Secure Socket Layer
 - Heute veraltet und durch TLS ersetzt

gRPC vs REST API

gRPC	REST
Protocol buffers – Binärformat (kompakt & schnell)	JSON – XML-basiert (größere Nachrichten & langsamer)
HTTP/2 (lower latency)	Ursprünglich nur HTTP/1.1 (higher latency); jetzt aber auch HTTP/2
Bidirektional & a/synchron	Nur Client-Server-Requests
Streaming möglich	Nur Request/Response
Freie API-Gestaltung möglich	Orientiert an HTTP-Requests (GET, PUT, PUT, ...)

5.5 Zusammenfassung

Zusammenfassung

- Fernaufrufe (RPC) sind flexibel und viel einfacher zu verwenden, als Sockets
- Vorsicht ist bei Referenz-Parameter angesagt, falls diese automatisch serialisiert werden. Hier können u.U. sehr viele Daten übertragen werden was langsam ist.
- REST und gRPC sind weit verbreitet genutzt in verteilten Systemen